Tema 4: Protocolos de capa de aplicación

Actividades 4.1, 4.2 y 4.3. Servidor SSH, fail2ban y CheckTLS

Protocolos de Seguridad · Tema 4 · 2026/2027



Realizado por Miguel Bolívar García y Javier Francisco Dibo Gómez

Índice de contenidos

1. Actividad 4.1: Servidor SSH

2. Actividad 4.2: fail2ban

- 2.1 Entorno
- 2.2 Instalación
- 2.3 Configuración
- 2.4 Modo del filtro sshd
- 2.5 Ataque y bloqueo
- 2.6 Desbloqueo
- 2.7 Bots de Internet
- 2.8 Mejoras sin activar

3. Actividad 4.3: CheckTLS

- 3.1 Cuentas analizadas
- 3.2 Bloqueo de la prueba en el navegador
- 3.3 Método
- 3.4 Resultado de Gmail
- 3.5 Resultado de Mailinator
- 3.6 Comparación

4. Referencias

Índice de figuras

Figura 2.1: Versión del sistema, paquetes instalados y estado del servicio

Figura 2.2: jail.local y los valores que usa la jaula sshd en ejecución

Figura 2.3: Los intentos 1 a 5 llegan a sshd. El 6.º recibe Connection refused

Figura 2.4: Estado de la jaula y tabla de nftables durante el bloqueo

Figura 2.5: El portátil entra en Act1 a las 17:16:39, con Act2 bloqueada

Figura 2.6: Act2 vuelve a llegar a sshd a las 17:21:11

Figura 2.7: Registro de fail2ban: 5 fallos, bloqueo, desbloqueo y el fallo del intento nuevo

Figura 3.1: CheckTLS en Firefox: aviso de la suscripción y Test Results vacío

Figura 3.2: Resultado de gmail.com: 116 de 116 y PASS en todas las comprobaciones de los 5 MX

Figura 3.3: Detalle de gmail-smtp-in.l.google.com: TLS 1.3 y certificado validado

Figura 3.4: Resultado de mailinator.com: 111 de 116, con WARN en Secure y Cert

Figura 3.5: Detalle de mail.mailinator.com: certificado autofirmado y nombre que no coincide

Figura 3.6: Detalle de mail2.mailinator.com: certificado autofirmado y caducado

Índice de tablas

Tabla 2.1: equipos de la prueba

Tabla 2.2: directivas de jail.local

Tabla 2.3: modos normal y aggressive del filtro sshd

Tabla 2.4: cronología del registro de fail2ban

Tabla 3.1: dominios analizados

Tabla 3.2: comparación de gmail.com y mailinator.com en CheckTLS

Índice de bloques de código

Bloque de código 2.1: instalación de fail2ban en Act1

Bloque de código 2.2: configuración de fail2ban en Act1

Bloque de código 2.3: ataque desde Act2

Bloque de código 2.4: ejemplo de las dos mejoras (no aplicado)

Bloque de código 3.1: envío del formulario de TestReceiver con curl

1. Actividad 4.1: Servidor SSH

2. Actividad 4.2: fail2ban

fail2ban lee los registros de un servicio y cuenta los fallos de cada IP. Cuando una IP llega a `maxretry` fallos dentro de `findtime` segundos, la bloquea en el cortafuegos durante `bantime` segundos [1].

2.1 Entorno

Las pruebas se hicieron el 5 de noviembre de 2026 con las dos instancias de AWS del Tema 3.

Papel	Equipo	Sistema	IP pública
Servidor	Act1 (`JavierDiboAct1`)	RHEL 10.2	184.198.103.83
Atacante	Act2 (`JavierDiboAct2`)	RHEL 10.2	3.90.46.118
Administrador	Portátil	Ubuntu	207.188.144.145

Tabla 2.1: equipos de la prueba

En Act1, `sshd` solo acepta claves públicas. La instancia no tiene `firewalld` ni `iptables`, así que el bloqueo se hace con `nftables`.

2.2 Instalación

fail2ban no está en los repositorios de RHEL. Sale de EPEL 10 [2]. Se instala el paquete `fail2ban-server` y no el metapaquete `fail2ban`, que arrastra también `firewalld` y `sendmail`. `fail2ban-selinux` entra como dependencia.

terminal

```
sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-10.noarch.rpm
sudo dnf install -y fail2ban-server
sudo systemctl enable --now fail2ban
```

Bloque de código 2.1: instalación de fail2ban en Act1



Figura 2.1: Versión del sistema, paquetes instalados y estado del servicio

2.3 Configuración

`jail.conf` trae los valores por defecto y no se edita, porque una actualización lo sobrescribe. Los cambios van en `jail.local`, que fail2ban lee después [1].

/etc/fail2ban/jail.local

```
[DEFAULT]
bantime  = 300
findtime  = 600
maxretry  = 5
banaction = nftables-multiport
banaction_allports = nftables-allports
ignoreip  = 127.0.0.1/8 ::1 207.188.144.145

[sshd]
enabled = true
backend = systemd
```

Bloque de código 2.2: configuración de fail2ban en Act1

Directiva	Valor	Efecto
`maxretry`	5	Fallos que provocan el bloqueo
`findtime`	600	Ventana, en segundos, en la que se cuentan los fallos
`bantime`	300	Duración del bloqueo, en segundos
`banaction`	`nftables-multiport`	Bloquea los puertos de la jaula. `jail.conf` usa `iptables-multiport`, que no existe en Act1
`banaction_allports`	`nftables-allports`	La misma sustitución para las jaulas que bloquean todos los puertos
`ignoreip`	`127.0.0.1/8 ::1 207.188.144.145`	IP que nunca se bloquean. Incluye el portátil, para no perder el acceso por un error
`enabled`	`true`	Activa la jaula `sshd`
`backend`	`systemd`	Lee los mensajes de `sshd` en el journal de systemd, y no en un archivo de log

Tabla 2.2: directivas de jail.local

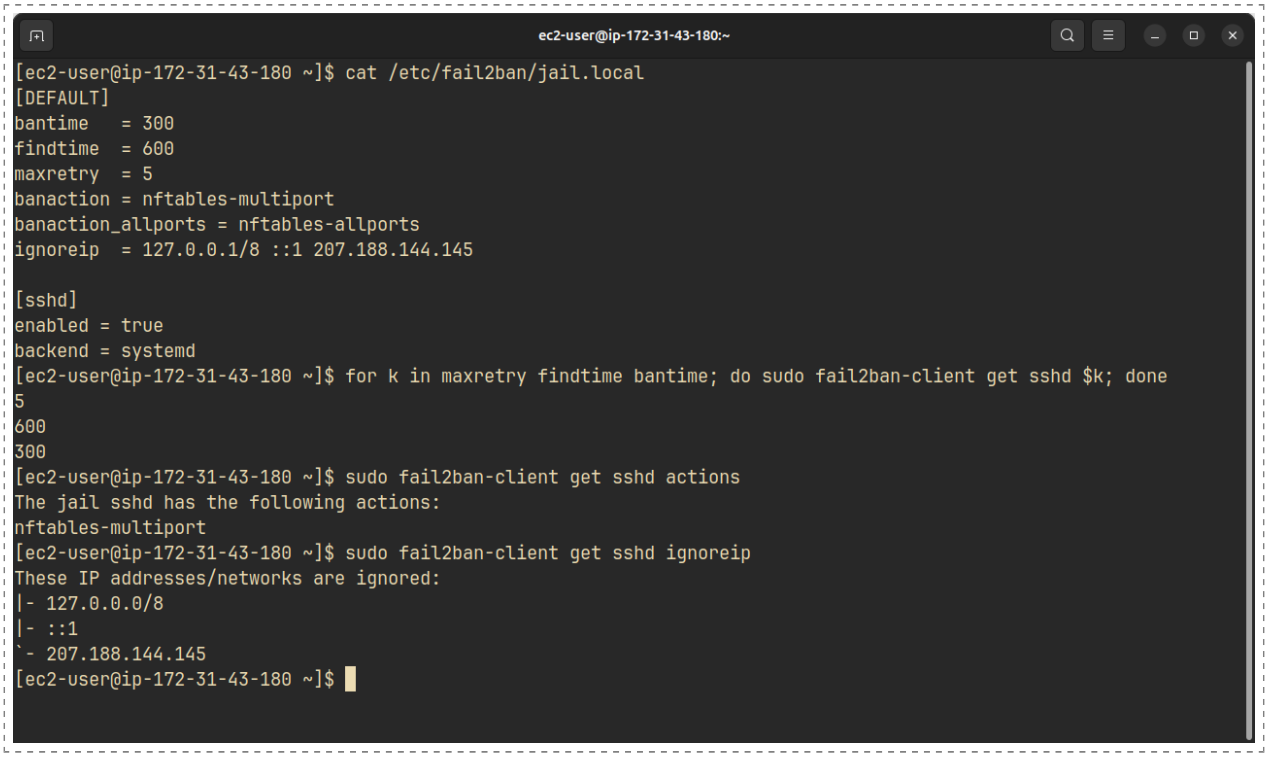


Figura 2.2: jail.local y los valores que usa la jaula sshd en ejecución

2.4 Modo del filtro `sshd`

El filtro `sshd` tiene varios modos [3]. La jaula usa `normal`, el modo por defecto.

	`normal`	`aggressive`
Conexión con un usuario que no existe	1 fallo (`Invalid user`)	2 fallos (también `Connection closed … [preauth]`)
Bloqueo con `maxretry = 5`	En la 5.ª conexión	En la 3.ª conexión
Conexión con un usuario real sin la clave correcta	No cuenta	Cuenta (`Connection closed … [preauth]`)
Escaneos sin autenticación (`Did not receive identification string`)	No cuenta	Cuenta

Tabla 2.3: modos normal y aggressive del filtro sshd

En `normal`, la línea `Connection closed … [preauth]` lleva la marca `<F-NOFAIL>`: identifica la IP, pero no suma un fallo [3]. Un usuario real sin la clave correcta no puede

entrar, porque el servidor no acepta contraseñas. Por eso el modo `normal` basta en Act1, y cada conexión del atacante cuenta exactamente un fallo.

2.5 Ataque y bloqueo

Desde Act2 se lanzan 6 conexiones seguidas con un usuario que no existe. `BatchMode=yes` evita que `ssh` pida nada.

terminal

```
for i in 1 2 3 4 5 6; do
echo "Intento $i, $(date +%T)"; ssh -o BatchMode=yes -o
ConnectTimeout=5 atacante@184.198.103.83
sleep 1; done
```

Bloque de código 2.3: ataque desde Act2



Figura 2.3: Los intentos 1 a 5 llegan a sshd. El 6.º recibe Connection refused

Los 5 primeros intentos llegan a `sshd` y fallan en la autenticación. El quinto fallo provoca el bloqueo, y el 6.º intento ya no llega al servicio: `nftables` rechaza el paquete con un ICMP `port-unreachable`, que el cliente muestra como `Connection refused`.

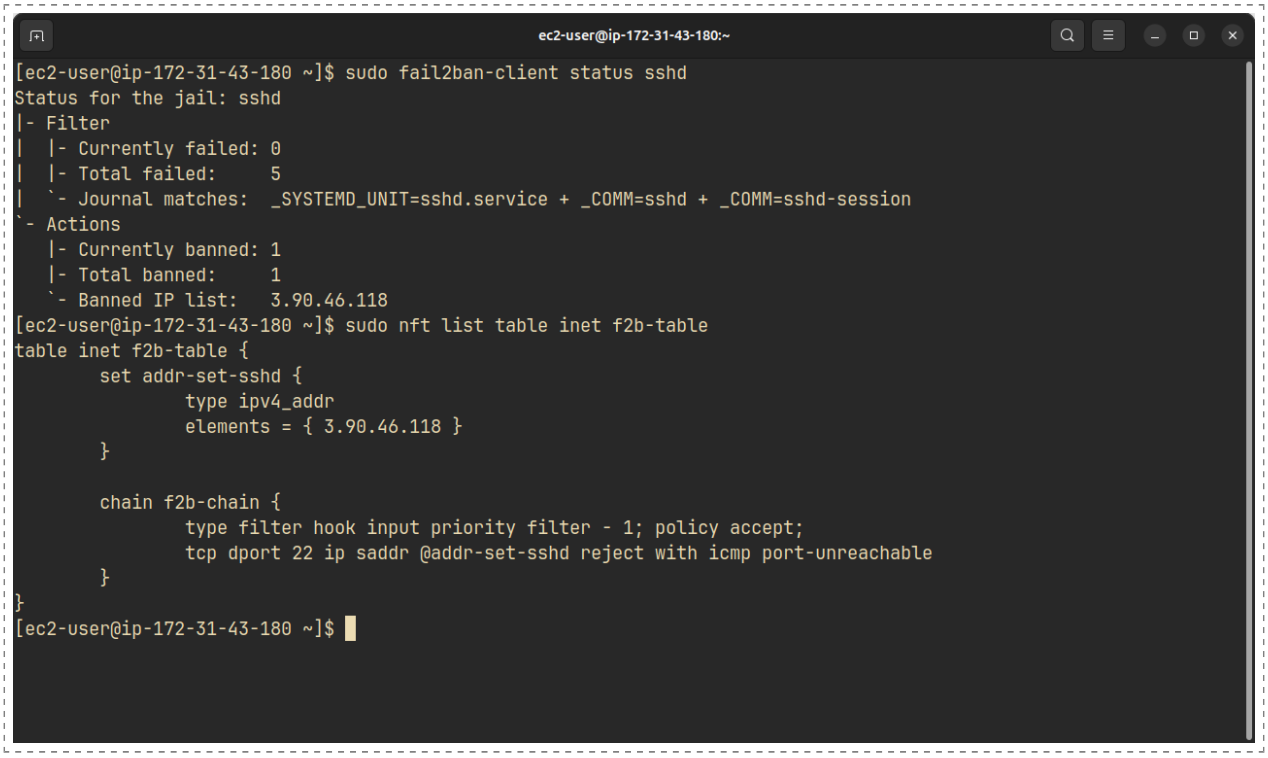


Figura 2.4: Estado de la jaula y tabla de nftables durante el bloqueo

La acción `nftables-multiport` crea la tabla `inet f2b-table` [4]. El conjunto `addr-set-sshd` guarda las IP bloqueadas, y la cadena `f2b-chain` rechaza el puerto 22 para esas IP. La regla solo afecta al puerto de la jaula: el resto de servicios de Act1 sigue accesible para el atacante.

Durante el bloqueo, el portátil entra en Act1 sin problemas, porque su IP está en `ignoreip`.



Figura 2.5: El portátil entra en Act1 a las 17:16:39, con Act2 bloqueada

2.6 Desbloqueo

A los 300 s, fail2ban saca la IP del conjunto. Un intento nuevo desde Act2 vuelve a llegar a `sshd`, y recibe otra vez `Permission denied`.



Figura 2.6: Act2 vuelve a llegar a sshd a las 17:21:11

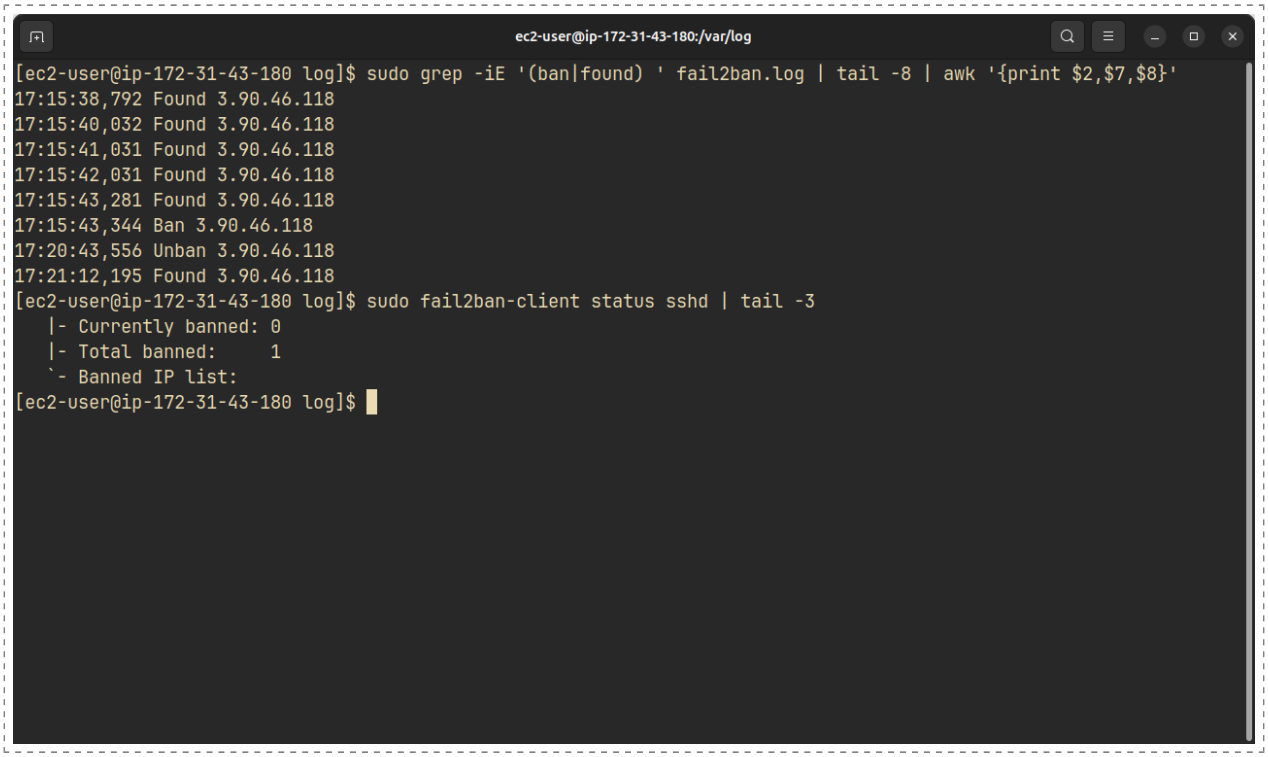


Figura 2.7: Registro de fail2ban: 5 fallos, bloqueo, desbloqueo y el fallo del intento nuevo

Hora	Evento
17:15:38,792 a 17:15:43,281	5 fallos (`Found`) de 3.90.46.118
17:15:43,344	`Ban` , 63 ms después del quinto fallo
17:20:43,556	`Unban` , 300 s después del bloqueo
17:21:12,195	`Found` del intento nuevo

Tabla 2.4: cronología del registro de fail2ban

Los 5 fallos caen en 4,5 s, muy dentro de los 600 s de `findtime` . Tras el desbloqueo, `Currently banned` vuelve a 0 y `Total banned` queda en 1.

2.7 Bots de Internet

Un servidor SSH con una IP pública recibe intentos de bots que prueban usuarios y contraseñas. La misma jaula los bloquearía igual que a Act2. Durante la prueba no llegó ninguno: el registro de fail2ban y el journal de `sshd` solo contienen las IP de Act2 y del portátil, y el único bloqueo fue el de Act2.

2.8 Mejoras sin activar

Dos opciones de fail2ban endurecen el bloqueo de una IP que reincide [1]. No se activan, porque cambian los 300 s que pide el enunciado.

- **`bantime.increment = true`** : cada bloqueo nuevo de la misma IP dura más que el anterior. Con `bantime.factor` y `bantime.maxtime` se controla el crecimiento y el máximo.
- **La jaula `recidive`** : lee el propio registro de fail2ban. Una IP bloqueada varias veces en un periodo largo recibe un bloqueo largo en todos los puertos, con la acción de `banaction_allports` .

```
[DEFAULT]
bantime.increment = true
bantime.factor    = 2
bantime.maxtime   = 1w

[recidive]
enabled  = true
logpath  = /var/log/fail2ban.log
bantime  = 1w
findtime = 1d
maxretry = 3
```

Bloque de código 2.4: ejemplo de las dos mejoras (no aplicado)

3. Actividad 4.3: CheckTLS

CheckTLS TestReceiver [5] se conecta a los servidores de correo (MX) de un dominio y comprueba la sesión SMTP con STARTTLS [6]: la versión de TLS, el cifrado y el certificado de cada MX.

3.1 Cuentas analizadas

Cuenta	Dominio	Motivo
Correo limpio	`gmail.com`	Proveedor grande con certificados de una CA pública
Correo con deficiencias	`mailinator.com`	Correo desechable. Sus MX usan certificados autofirmados

Tabla 3.1: dominios analizados

La cuenta de la Universidad de Jaén no aporta nada distinto: sus MX son los de Google, los mismos que analiza `gmail.com`. El test solo necesita el dominio, así que no se envía ninguna dirección personal.

3.2 Bloqueo de la prueba en el navegador

La página no devuelve resultados en el navegador. Al pulsar *Run Test*, el campo del dominio se marca en rojo, la sección *Test Results* queda vacía y aparece el aviso: "To try CheckTLS without a subscription please turn off tracking prevention and ad blocking so we can count your tests."

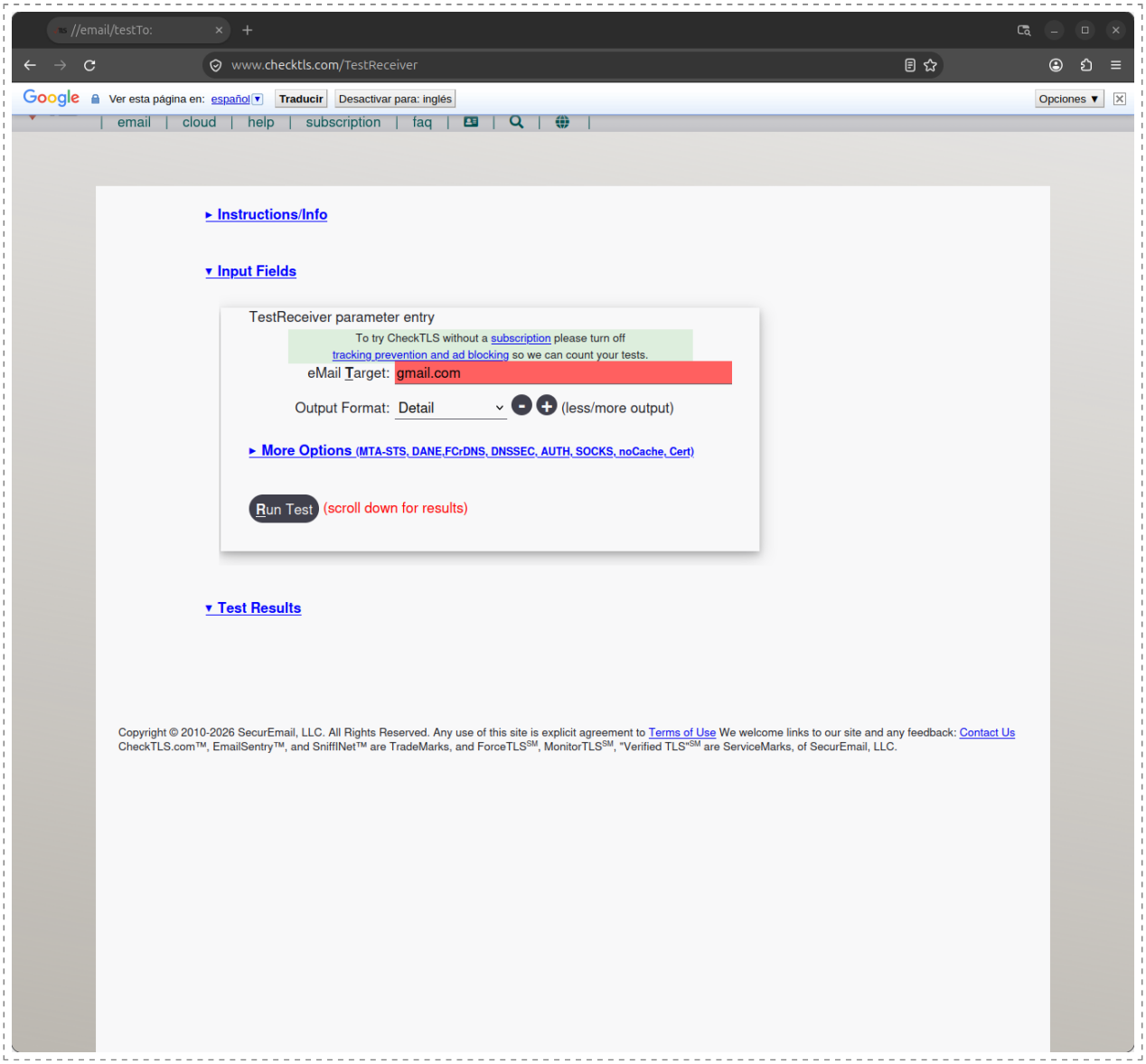


Figura 3.1: CheckTLS en Firefox: aviso de la suscripción y Test Results vacío

Se probó en dos navegadores de escritorio y en un móvil con datos móviles, fuera de la red de casa, con el mismo resultado. La versión con los parámetros en la URL tampoco funciona sin cuenta: responde "Use of CheckTLS web services requires COMPANYCODE and COMPANYPASS".

3.3 Método

El resultado se obtiene enviando el mismo formulario con `curl`. La primera petición descarga la página, con su cookie y su `csrf_token`. La segunda envía el formulario con el dominio y el formato *Detail* (`LEVEL=4`), como haría el botón *Run Test*.

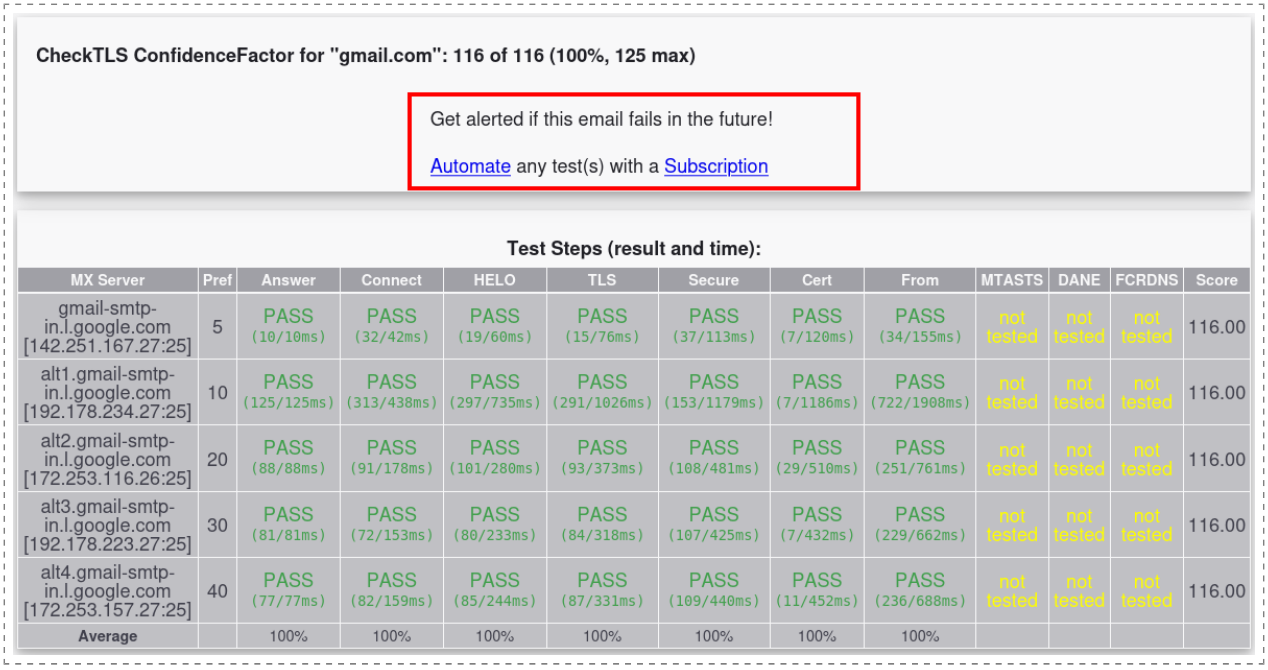
terminal

```
for d in gmail.com mailinator.com; do
  curl -s -c cj -b cj -A 'Mozilla/5.0' https://www.checktls.com/
TestReceiver -o form.html
  tok=$(grep -oP 'name="csrf_token" value="\K[^"]*' form.html)
  curl -s -c cj -b cj -A 'Mozilla/5.0' -e https://www.checktls.com/
TestReceiver \
    --data-urlencode "csrf_token=$tok" --data-urlencode EMAIL=$d \
    -d LEVEL=4 -d PROGRESS=off -d QUICK=off \
    https://www.checktls.com/TestReceiver -o ${d%%.*}.html --max-time
170
done
```

Bloque de código 3.1: envío del formulario de TestReceiver con curl

La respuesta es la página de CheckTLS con la sección *Test Results* completa. Las figuras de esta actividad son esa página, abierta en Firefox con la hoja de estilos de CheckTLS. El texto completo de cada resultado está en `lab/checktls-gmail.txt` y `lab/checktls-mailinator.txt`.

3.4 Resultado de Gmail



CheckTLS ConfidenceFactor for "gmail.com": 116 of 116 (100%, 125 max)

Get alerted if this email fails in the future!

Automate any test(s) with a Subscription

Test Steps (result and time):

MX Server	Pref	Answer	Connect	HELO	TLS	Secure	Cert	From	MTASTS	DANE	FCRDNS	Score
gmail-smtp-in.l.google.com [142.251.167.27:25]	5	PASS (10/10ms)	PASS (32/42ms)	PASS (19/60ms)	PASS (15/76ms)	PASS (37/113ms)	PASS (7/120ms)	PASS (34/155ms)	not tested	not tested	not tested	116.00
alt1.gmail-smtp-in.l.google.com [192.178.234.27:25]	10	PASS (125/125ms)	PASS (313/438ms)	PASS (297/735ms)	PASS (291/1026ms)	PASS (153/1179ms)	PASS (7/1186ms)	PASS (722/1908ms)	not tested	not tested	not tested	116.00
alt2.gmail-smtp-in.l.google.com [172.253.116.26:25]	20	PASS (88/88ms)	PASS (91/178ms)	PASS (101/280ms)	PASS (93/373ms)	PASS (108/481ms)	PASS (29/510ms)	PASS (251/761ms)	not tested	not tested	not tested	116.00
alt3.gmail-smtp-in.l.google.com [192.178.223.27:25]	30	PASS (81/81ms)	PASS (72/153ms)	PASS (80/233ms)	PASS (84/318ms)	PASS (107/425ms)	PASS (7/432ms)	PASS (229/662ms)	not tested	not tested	not tested	116.00
alt4.gmail-smtp-in.l.google.com [172.253.157.27:25]	40	PASS (77/77ms)	PASS (82/159ms)	PASS (85/244ms)	PASS (87/331ms)	PASS (109/440ms)	PASS (11/452ms)	PASS (236/688ms)	not tested	not tested	not tested	116.00
Average		100%	100%	100%	100%	100%	100%	100%				

Figura 3.2: Resultado de gmail.com: 116 de 116 y PASS en todas las comprobaciones de los 5 MX

Los 5 MX de Gmail pasan todas las comprobaciones, con 116 puntos cada uno. La Figura 3.3 muestra el detalle del MX principal. La sesión negocia TLS 1.3 con `TLS_AES_256_GCM_SHA384` y secreto perfecto hacia delante. El certificado es válido (`Cert VALIDATED: ok`), lo emite Google Trust Services (WE2) y su `subjectAltName`

incluye `gmail-smtp-in.l.google.com`. Tras STARTTLS, el segundo `EHLO` ya no anuncia `STARTTLS`.

```
seconds    Test Steps and Results
[000.001]  Trying TLS on gmail-smtp-in.l.google.com[142.251.167.27:25] (5) @2026-11-05T17:35:44.980266Z (S)
[000.010]  Server answered
[000.041]<-- 220 mx.google.com ESMTP d75a77b69052e-54dd695cf09si5930241cf.272 - gsmtp
[000.042]  We are allowed to connect
[000.042]--> EHLO www12-do.checktls.com
[000.060]<-- 250-mx.google.com at your service, [142.93.73.156]
           250-SIZE 157286400
           250-8BITMIME
           250-STARTTLS
           250-ENHANCEDSTATUSCODES
           250-PIPELINING
           250-CHUNKING
           250-SMTPUTF8
           250 LIMITS MAILMAX=1000 RCPTMAX=100 RCPTDOMAINMAX=1
[000.060]  We can use this server
[000.060]  TLS is an option on this server
[000.061]--> STARTTLS
[000.075]<-- 220 2.0.0 Ready to start TLS
[000.076]  STARTTLS command works on this server
[000.076]  SSL_ocsp_mode = SSL_OCSP_FULL_CHAIN
[000.113]  Connection converted to SSL/TLS
           SSLVersion in use: TLSv1_3
           TLS_AES_256_GCM_SHA384 (quality:5 on a scale of 0 to 5)
           Perfect Forward Secrecy: yes
           Session Algorithm in use: X25519(253 bits)
           Key Exchange Quality: good (quality:4 on a scale of 0 to 5)
           Certificate #1 of 4 (sent by MX):
           Cert VALIDATED: ok
           Cert Hostname VERIFIED (gmail-smtp-in.l.google.com = mx.google.com | DNS:mx.google.com | DNS:smtp.google.com | DNS:aspmx.l.google.com |
           DNS:alt1.aspmx.l.google.com | DNS:alt2.aspmx.l.google.com | DNS:alt3.aspmx.l.google.com | DNS:alt4.aspmx.l.google.com | DNS:gmail-smtp-in.l.google.com
           | DNS:alt1.gmail-smtp-in.l.google.com | DNS:alt2.gmail-smtp-in.l.google.com | DNS:alt3.gmail-smtp-in.l.google.com | DNS:alt4.gmail-smtp-
           in.l.google.com | DNS:gmr-smtp-in.l.google.com | DNS:alt1.gmr-smtp-in.l.google.com | DNS:alt2.gmr-smtp-in.l.google.com | DNS:alt3.gmr-smtp-
           in.l.google.com | DNS:alt4.gmr-smtp-in.l.google.com | DNS:mx1.smtp.goog | DNS:mx2.smtp.goog | DNS:mx3.smtp.goog | DNS:mx4.smtp.goog |
           DNS:aspmx2.googlemail.com | DNS:aspmx3.googlemail.com | DNS:aspmx4.googlemail.com | DNS:aspmx5.googlemail.com | DNS:gmr-mx.google.com)
           cert not revoked by OCSP
           Not Valid Before: Oct  7 19:09:08 2026 GMT
           Not Valid After: Dec 30 19:09:07 2026 GMT
           Seconds Until Expired: 4757601
           subject: /CN=mx.google.com
           issuer: /C=US/O=Google Trust Services/CN=WE2
```

Figura 3.3: Detalle de gmail-smtp-in.l.google.com: TLS 1.3 y certificado validado

Las columnas MTA-STS [7], DANE [8] y FCrDNS salen como *not tested*. Son comprobaciones de *More Options* que el test no activa por defecto.

3.5 Resultado de Mailinator

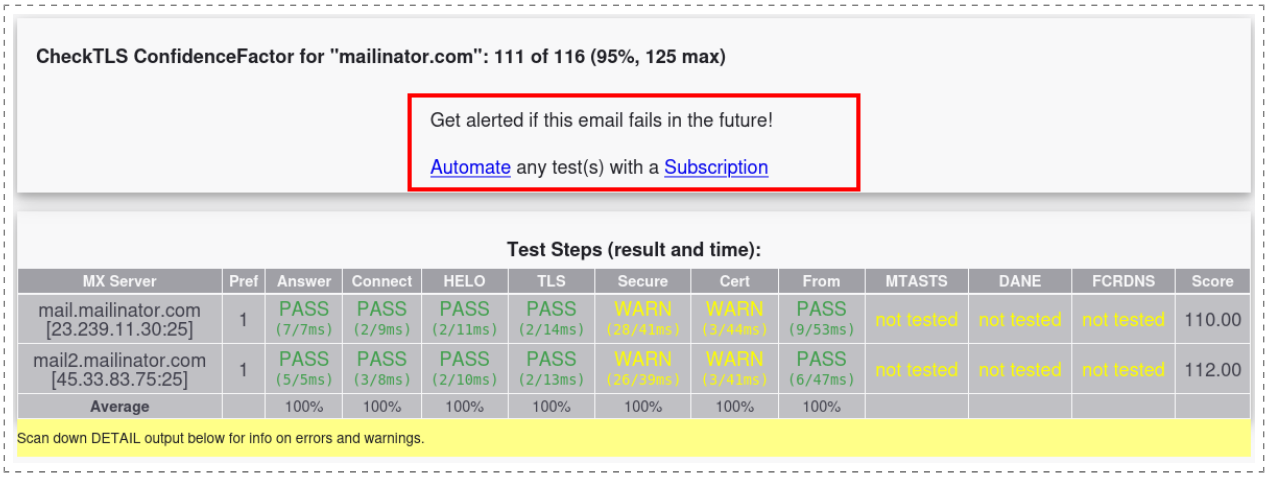


Figura 3.4: Resultado de mailinator.com: 111 de 116, con WARN en Secure y Cert

Los 2 MX de Mailinator conectan y cifran con TLS 1.3, pero fallan la validación del certificado. Las columnas *Secure* y *Cert* dan WARN.

```
seconds           Test Steps and Results
[000.001]         Trying TLS on mail.mailinator.com[23.239.11.30:25] (1) @2026-11-05T17:35:51.282906Z (S)
[000.007]         Server answered
[000.009]    <--  220 Mailinator.com ESMTP MailinatorSMTP
[000.009]         We are allowed to connect
[000.009]    -->  EHLO www11-do.CheckTLS.com
[000.011]    <--  250-mail.mailinator.com
                  250-8BITMIME
                  250-STARTTLS
                  250 Ok
[000.011]         We can use this server
[000.011]         TLS is an option on this server
[000.012]    -->  STARTTLS
[000.013]    <--  220 Ready to start TLS
[000.014]         STARTTLS command works on this server
[000.014]         SSL_ocsp_mode = SSL_OCSP_FULL_CHAIN
[000.041]         Connection converted to SSL/TLS
                  SSLVersion in use: TLSv1_3
                  TLS_AES_256_GCM_SHA384 (quality:5 on a scale of 0 to 5)
                  Perfect Forward Secrecy: yes
                  Session Algorithm in use: X25519(253 bits)
                  Key Exchange Quality: good (quality:4 on a scale of 0 to 5)
                  Certificate #1 of 1 (sent by MX):
                  Cert VALIDATION ERROR(S): self signed certificate
                  So email is encrypted but the recipient domain is not verified
                  Cert Hostname DOES NOT VERIFY (mail.mailinator.com != Unknown)
                  (turning on Send SNI might fix this: Try It)
                  So email is encrypted but the host is not verified
                  cert not revoked by OCSP
                  Not Valid Before: Oct 21 15:03:21 2019 GMT
                  Not Valid After: Aug  4 15:03:21 2293 GMT
                  Seconds Until Expired: 8417683649
                  subject: /C=US/ST=Unknown/L=Unknown/O=Mailinator/OU=Mailinator/CN=Unknown
                  issuer: /C=US/ST=Unknown/L=Unknown/O=Mailinator/OU=Mailinator/CN=Unknown
[000.046]    ~~>  EHLO www11-do.CheckTLS.com
[000.050]    <~~  250-mail.mailinator.com
                  250-8BITMIME
                  250-STARTTLS
                  250 Ok
[000.050]         SSL/TLS is working correctly on this server
[000.050]         warning: STARTTLS after second EHLO (see RFC3207 #4.2)
```

Figura 3.5: Detalle de mail.mailinator.com: certificado autofirmado y nombre que no coincide

`mail.mailinator.com` presenta un certificado autofirmado (`Cert VALIDATION ERROR(S): self signed certificate`) con `CN=Unknown`. El nombre no coincide con el del servidor (`Cert Hostname DOES NOT VERIFY`). El certificado es válido de 2019 a 2293, un periodo que ninguna CA pública emite.

```
seconds           Test Steps and Results
[000.001]         Trying TLS on mail2.mailinator.com[45.33.83.75:25] (1) @2026-11-05T17:35:51.341013Z (S)
[000.005]         Server answered
[000.008]    <--  220 Mailinator.com ESMTP MailinatorSMTP
[000.008]         We are allowed to connect
[000.008]    -->  EHLO www11-do.CheckTLS.com
[000.010]    <--  250-mail.mailinator.com
                  250-8BITMIME
                  250-STARTTLS
                  250 Ok
[000.010]         We can use this server
[000.010]         TLS is an option on this server
[000.010]    -->  STARTTLS
[000.012]    <--  220 Ready to start TLS
[000.013]         STARTTLS command works on this server
[000.013]         SSL_ocsp_mode = SSL_OCSP_FULL_CHAIN
[000.039]         Connection converted to SSL/TLS
                  SSLVersion in use: TLSv1_3
                  TLS_AES_256_GCM_SHA384 (quality:5 on a scale of 0 to 5)
                  Perfect Forward Secrecy: yes
                  Session Algorithm in use: X25519(253 bits)
                  Key Exchange Quality: good (quality:4 on a scale of 0 to 5)
                  Certificate #1 of 1 (sent by MX): EXPIRED
                  Cert VALIDATION ERROR(S): self signed certificate; certificate has expired
                  So email is encrypted but the recipient domain is not verified
                  Cert Hostname VERIFIED (mail2.mailinator.com = mail2.mailinator.com)
                  cert not revoked by OCSP
```

Figura 3.6: Detalle de mail2.mailinator.com: certificado autofirmado y caducado

`mail2.mailinator.com` presenta otro certificado autofirmado, que además caducó el 23 de marzo de 2025 (`certificate has expired`). En este caso el nombre sí coincide.

CheckTLS resume las dos situaciones igual: "So email is encrypted but the recipient domain is not verified". El correo viaja cifrado, pero el emisor no puede saber si habla con el servidor real de Mailinator o con un intermediario. Un atacante en la ruta puede presentar su propio certificado autofirmado sin que el emisor note la diferencia.

Los dos MX tienen otro aviso: `warning: STARTTLS after second EHLO (see RFC3207 #4.2)`. El servidor vuelve a anunciar `STARTTLS` en la respuesta al `EHLO` que sigue al handshake, y la RFC 3207 lo prohíbe [6, §4.2].

3.6 Comparación

	`gmail.com`	**`mailinator.com`**
ConfidenceFactor	116 de 116 (100 %)	111 de 116 (95 %)
MX analizados	5	2
Versión de TLS	TLS 1.3	TLS 1.3
Cifrado	`TLS_AES_256_GCM_SHA384`	`TLS_AES_256_GCM_SHA384`
Emisor del certificado	Google Trust Services	El propio servidor (autofirmado)
Validación del certificado	Correcta	Error en los 2 MX
Nombre del certificado	Coincide	No coincide en `mail`, coincide en `mail2`
Caducidad	En vigor	`mail2` caducado desde el 23-mar-2025
`STARTTLS` tras el segundo `EHLO`	No	Sí, en los 2 MX

Tabla 3.2: comparación de gmail.com y mailinator.com en CheckTLS

El cifrado de las dos sesiones es idéntico. La diferencia está en la autenticación del servidor: Gmail prueba su identidad con una cadena de confianza pública, y Mailinator no puede probarla. La puntuación solo baja 5 puntos, pero la deficiencia es seria. Con STARTTLS oportunista, la mayoría de los emisores SMTP aceptan un certificado no válido, así que no detectan a un intermediario [8].

4. Referencias

[1] The Fail2Ban Project, "jail.conf", *fail2ban*, GitHub. [En línea]. Disponible: https://github.com/fail2ban/fail2ban/blob/master/config/jail.conf. [Accedido: 5-nov-2026].

[2] Fedora Project, "Extra Packages for Enterprise Linux (EPEL)", *Fedora Docs*. [En línea]. Disponible: https://docs.fedoraproject.org/en-US/epel/. [Accedido: 5-nov-2026].

[3] The Fail2Ban Project, "filter.d/sshd.conf", *fail2ban* 1.1.0, GitHub. [En línea]. Disponible: https://github.com/fail2ban/fail2ban/blob/master/config/filter.d/sshd.conf. [Accedido: 5-nov-2026].

[4] The Fail2Ban Project, "action.d/nftables.conf", *fail2ban*, GitHub. [En línea]. Disponible: https://github.com/fail2ban/fail2ban/blob/master/config/action.d/nftables.conf. [Accedido: 5-nov-2026].

[5] SecurEmail, LLC, "CheckTLS TestReceiver". [En línea]. Disponible: https://www.checktls.com/TestReceiver. [Accedido: 5-nov-2026].

[6] P. Hoffman, "SMTP Service Extension for Secure SMTP over Transport Layer Security", RFC 3207, IETF, feb. 2002. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc3207. [Accedido: 5-nov-2026].

[7] D. Margolis, M. Risher, B. Ramakrishnan, A. Brotman y J. Jones, "SMTP MTA Strict Transport Security (MTA-STS)", RFC 8461, IETF, sep. 2018. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc8461. [Accedido: 5-nov-2026].

[8] V. Dukhovni y W. Hardaker, "SMTP Security via Opportunistic DNS-Based Authentication of Named Entities (DANE) Transport Layer Security (TLS)", RFC 7672, IETF, oct. 2015. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc7672. [Accedido: 5-nov-2026].